

Detalhamento técnico

Sistema de emissão de Notas Fiscais — Angular 19 + ASP.NET Core / C# (.NET 10) + PostgreSQL. Dois microsserviços: Estoque e Faturamento.

Estoque :5081

Faturamento :5082

App :4200

PostgreSQL :5433

Angular Material 19

Tema claro e escuro

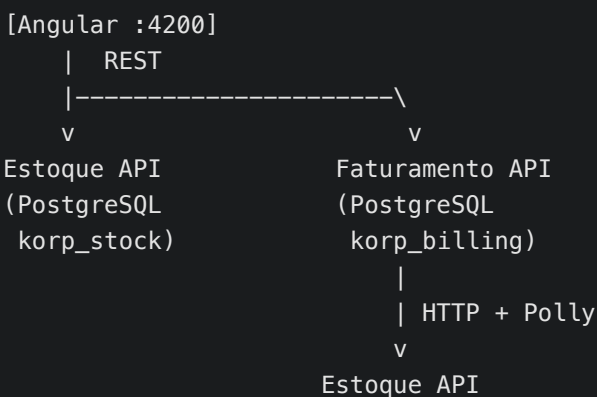
1. Visão da solução

O frontend Angular consome duas APIs REST independentes, cada uma com o próprio banco:

- **Korp.Stock.Api** (:5081): cadastro de produtos, consulta de saldo, baixa/estorno com lock de concorrência e modo de falha simulada.
- **Korp.Billing.Api** (:5082): numeração sequencial de notas, itens, impressão, compensação e assistente.

Pastas: frontend/ , microservices/stock-api , microservices/billing-api , infra/ , scripts/ .

A baixa de estoque **não** ocorre no cadastro da nota. Ocorre só na **impressão**. Se o estoque falhar no meio, o faturamento estorna as baixas já feitas e a nota permanece **Aberta**.



2. Quais ciclos de vida do Angular foram utilizados

Hook	Onde	Para quê
ngOnInit	Shell, Pannel, Produtos, Notas, editor	Cargas HTTP, busca com debounce, health checks e breakpoint do drawer.

<code>ngAfterViewInit</code>	Produtos e editor da nota	Foco no primeiro campo ou botão.
<code>ngOnChanges</code>	<code>InvoiceItemRowComponent</code>	Atualiza o saldo visível quando o produto do item muda.
<code>ngOnDestroy</code>	Shell, Painei, Produtos, editor	Cancela Subscription / <code>takeUntil</code> .

Componentes **standalone**. Rotas com `loadComponent` (lazy loading).

3. Se foi feito uso da biblioteca RxJS e, em caso afirmativo, como

Sim. O RxJS é o modelo de assincronismo do frontend.

- `switchMap` + `debounceTime` + `distinctUntilChanged` : busca de produtos.
- `interval` + `startWith` + `switchMap` + `catchError` : polling de saúde a cada 5 s.
- `combineLatest` : o Painei junta produtos, notas e insights.
- `exhaustMap` : o botão Imprimir ignora cliques repetidos.
- `finalize` : encerra o spinner da impressão.
- `Subject` / `messages$` : canal do `ToastService`, exibido em `MatSnackBar`.
- `takeUntil` : descadastro no `ngOnDestroy`.
- `BreakpointObserver` (CDK): alterna o drawer entre `side` e `over` .
- Interceptor: `catchError` lê `ProblemDetails` e mostra o `detail` .

4. Quais outras bibliotecas foram utilizadas e para qual finalidade

- **Angular Material 19** + CDK: interface completa em Material Design 3, tema e breakpoints.
- Roboto e Material Icons (Google Fonts).
- NuGet: EF Core + Npgsql, Swashbuckle, Microsoft.Extensions.Http.Resilience.
- Testes: xUnit, Testcontainers (PostgreSQL), Jasmine/Karma.

Não há Bootstrap, Tailwind nem CSS de ERP paralelo.

5. Para componentes visuais, quais bibliotecas foram utilizadas

Angular Material 19 (Material Design 3):

- `MatSidenav` , `MatToolbar` , `MatNavList` , `MatIcon` — casco da aplicação.
- `MatCard` , `MatButton` , `MatFormField` , `MatInput` , `MatSelect` .
- `MatTable` , `MatChip` , `MatList` , `MatSnackBar` , `MatProgressSpinner` .
- `MatSlideToggle` + `MatTooltip` — troca de tema claro/escuro.

UX/UI: escala de 8 px, drawer de 280 px em `surface` com divisória de 1 px, item ativo em pill, app bar sticky de 64 px, transições de 200 ms com `prefers-reduced-motion` respeitado.

Tema claro e escuro: `mat.theme` com `theme-type: color-scheme` emite `light-dark()`. O `ThemeService` (signal) aplica a classe `theme-dark` no `html`, persiste em `localStorage` e usa `prefers-color-scheme` na primeira visita. Script inline no `index.html` evita flash de tema.

O preview da nota é HTML dentro de `mat-card`, com CSS de impressão. Não é NF-e SEFAZ.

6. Como foi realizado o gerenciamento de dependências no Golang (se aplicável)

Não aplicável. Backend em **C# / .NET 10**. Dependências via **NuGet** nos `.csproj`, `dotnet restore`. Não há `go.mod`.

7. Quais frameworks foram utilizados no Golang ou C#

- ASP.NET Core Web API (controllers + Swagger/Swashbuckle)
- Entity Framework Core 10 + Npgsql
- `Microsoft.Extensions.Http.Resilience` (Polly: retry, timeout, circuit breaker) só no `HttpClient` faturamento → estoque
- CORS restrito a `http://localhost:4200`

LINQ é `System.Linq`, não um framework separado.

8. Como foram tratados os erros e exceções no backend

1. Exceções de domínio (`AppException`) com código, título, detalhe e status HTTP.
2. Middleware global devolve **RFC 7807 ProblemDetails**.
3. `VALIDATION` 400, `NOT_FOUND` 404, `INSUFFICIENT_STOCK` / `CONFLICT` 409, `STOCK_UNAVAILABLE` 503, `INTERNAL` 500.
4. Na impressão, falha no estoque dispara compensação; a nota não fecha.
5. O frontend mostra o `detail` no snackbar.

Falha simulada: `POST /api/chaos`. Rotas de negócio passam a 503. `/health` e `/chaos` continuam no ar.

9. Caso a implementação utilize C#, indicar se foi utilizado LINQ e de que forma

- Produtos: `Where` + `OrderBy` + `Select` + `ToListAsync`.
- Notas: `OrderByDescending` + `Select`.
- Itens: `GroupBy` + `Sum`.
- Insights: `Where`, `SelectMany`, `GroupBy`.
- Compensação: `Enumerable.Reverse`.

A numeração sequencial incrementa `invoice_sequences` em transação. Não é `Max + 1`.

10. Requisitos opcionais

a) Concorrência

Débito com `pg_advisory_xact_lock(productId)` . Duas impressões no `PAR-M8` (saldo 1): uma fecha, a outra recebe 409. Script: `scripts/test-concurrency.sh` .

b) Inteligência artificial

Sem chave de LLM: sugestão de descrição no estoque, insights no Pannel, assistente por intenções no faturamento.

c) Idempotência

Reimpressão de nota Fechada sem nova baixa. `operationId = invoice:{id}:product:{id}` . Header `Idempotency-Key` . `exhaustMap` no botão.

11. Banco de dados e testes

PostgreSQL 16 (Docker, porta 5433). Bancos `korp_stock` e `korp_billing` . `EnsureCreated` + `seed`. Testes de integração no mesmo PostgreSQL via Testcontainers.

```
dotnet test Korp.Teste.slnx · cd frontend && npm test
```

No Angular: serviços HTTP, interceptor, `ngOnChanges` da linha de item, numeração da lista e o serviço de tema.

12. Como rodar a demonstração de falha

1. Crie uma nota Aberta.
2. No drawer, **Simular falha do estoque**.
3. **Imprimir nota fiscal**: spinner, snackbar, status continua Aberta.
4. **Restaurar estoque**, aguarde ~5 s (circuit breaker) e imprima de novo.

Documento de detalhamento técnico pedido no enunciado do teste Korp. Entregar com o repositório e o vídeo.